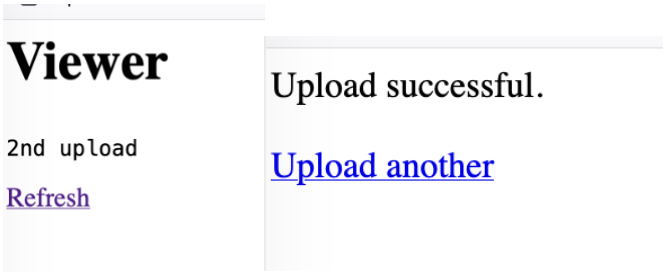
1.

```
== Saving stack outputs ==
== Deployment complete ==
Uploader: http://itmo-444-544-app-alb-2023687996.us-east-2.elb.amazonaws.com:8080/
Viewer:   http://itmo-444-544-app-alb-2023687996.us-east-2.elb.amazonaws.com:8081/
```

2.



3.

```
    --target-group-arn "$(grep '^UPLOADER_TG_ARN=' stack_outputs.env | cut -d= -f2-)" \
    --region us-east-2
{
    "TargetHealthDescriptions": [
        {
            "Target": {
                "Id": "i-09551dc6fd2650498",
                "Port": 3000
            },
            "HealthCheckPort": "3000",
            "TargetHealth": {
                "State": "healthy"
            },
            "AdministrativeOverride": {
                "State": "no_override",
                "Reason": "AdministrativeOverride.NoOverride",
                "Description": "No override is currently active on target"
            }
        }
    ]
}
root@8d53bccd4076:/cloud-computing-lab-test# aws elbv2 describe-target-health \
    --target-group-arn "$(grep '^VIEWER_TG_ARN=' stack_outputs.env | cut -d= -f2-)" \
    --region us-east-2
{
    "TargetHealthDescriptions": [
        {
            "Target": {
                "Id": "i-0a89287999458d9df",
                "Port": 3000
            },
            "HealthCheckPort": "3000",
            "TargetHealth": {
                "State": "healthy"
            },
            "AdministrativeOverride": {
                "State": "no_override",
                "Reason": "AdministrativeOverride.NoOverride",
                "Description": "No override is currently active on target"
            }
        }
    ]
}
```

4.
```
Poll log written to chaos_test_uploader_7876.csv
Recovery time: 83.0s (SLO: 180s)
PASS: recovery time met the SLO.
```

```
elapsed_seconds,is_up
0.0,True
2.06,False
0.0,False
2.05,False
4.11,False
6.16,False
8.22,False
10.27,False
12.33,False
```

```
Poll log written to chaos_test_viewer_8365.csv
Recovery time: 73.8s (SLO: 180s)
PASS: recovery time met the SLO.
```

```
elapsed_seconds,is_up
0.0,True
2.07,False
0.0,False
2.05,False
4.1,False
6.16,False
8.22,False
13.26,False
18.3,False
```

```
Waiting for the endpoint to go down...
Waiting for the endpoint to come back up...
Poll log written to chaos_test_uploader_8688.csv
Recovery time: 77.9s (SLO: 60s)
FAIL: recovery time exceeded the SLO.
```

```
elapsed_seconds,is_up
0.0,True
2.06,True
4.11,False
0.0,False
2.05,False
4.11,False
6.17,False
8.22,False
10.28,False
```

5. The chaos tests showed that both applications were able to recover after an instance failure. The uploader recovered in about 83 seconds, and the viewer recovered in about 74 seconds. Both passed the original 180-second recovery-time SLO. When I lowered the SLO to 60 seconds, the uploader took about 78 seconds to recover, so that test failed. I think most of the recovery time came from AWS detecting the failure, launching a replacement EC2 instance, starting the application, and waiting for the load balancer to recognize the new instance as healthy. The CSV files showed that the applications were unavailable for several checks before responding successfully again. One limitation I noticed is that the testing script starts a new timer when it begins checking for recovery, so the reported times do not include the entire process from the moment the instance was terminated.

6. In Lab 03, I used the deployment script to launch separate EC2 instances for the uploader and viewer applications. For Lab 04, I updated the script to use an Application Load Balancer, target groups, listeners, and Auto Scaling Groups across two Availability Zones. Instead of manually launching individual instances, the Auto Scaling Groups manage the instances and replace them if they fail. I also changed the security group settings so the applications only accept traffic from the load balancer on port 3000, rather than allowing direct access from the internet. The cleanup script also needed to be updated because there were more AWS resources to remove. I added steps to delete the Auto Scaling Groups, load balancer, target groups, and security group. I also used Python scripts to simulate instance failures, check when the applications became available again, and save the results in CSV files to compare against the recovery-time SLO.